

Projekat za kurs Računarska inteligencija

Minimum Dominating Set Problem

Student: Viktor Šević | **Mentor:** Stefan Kapunac

Avgust 2026.

1 Uvod

1.1 Definicija problema

Problem minimalnog dominirajućeg skupa (eng. *Minimum Dominating Set Problem*, MDS) predstavlja jedan od klasičnih problema kombinatorne optimizacije i teorije grafova. Problem se javlja u različitim oblastima u kojima je potrebno odabrati što manji skup objekata koji može da „pokrije“ ili kontroliše sve ostale objekte. Primeri primene uključuju projektovanje komunikacionih i računarskih mreža, postavljanje baznih stanica, društvene mreže, distribuirane sisteme i različite probleme pokrivanja u velikim mrežama.

Dominirajući skup D grafa G naziva se minimalnim dominirajućim skupom ako ne postoji pravi podskup $D' \subset D$ koji je dominirajući skup grafa G . Dominirajući skup najmanje kardinalnosti naziva se minimalnim dominirajućim skupom, a njegov kardinalni broj određuje (donji) dominacioni broj $\gamma(G)$ za dati graf G . [1]

Problem minimalnog dominantnog skupa je NP-težak, zbog čega se za velike instance ne očekuje postojanje efikasnog algoritma koji u svim slučajevima pronalazi optimalno rešenje. Zbog toga su razvijeni različiti egzaktni i heuristički pristupi za njegovo rešavanje, uključujući potpunu pretragu, metode grananja i odsecanja, celobrojno programiranje, lokalnu pretragu, genetske algoritme i različite metaheuristike.

1.2 Obrazloženje

U radu će se eksperimentalno uporediti navedene metode na grafovima različitih veličina i gustina. Kvalitet rešenja procenjuje se na osnovu odstupanja od optimalnog rešenja, dok se efikasnost algoritama procenjuje na osnovu vremena izvršavanja. Na manjim instancama rezultati heurističkih metoda porede se sa optimalnim rešenjima dobijenim egzaktnom metodom, dok se na većim instancama ispituje njihovo ponašanje kada egzaktno rešavanje postaje vremenski zahtevno. Posebna pažnja posvećena je poređenju VNS algoritma sa i bez *pruning*-a

1.3 Osvrt na literaturu

Već su poznata metaheuristička rešenja i dobro izučena njihova primena u radu Hedar i Ismail (2010), gde je pokazano da hibridni pristupi zasnovani na genetskim algoritmima i lokalnoj pretrazi pružaju izuzetnu efikasnost i brzo konvergiraju ka optimalnim rešenjima kod problema minimalnog dominirajućeg skupa. U njihovom istraživanju demonstrirano je da specijalizovani mehanizmi popravke i orezivanja (*repair i pruning/filtering*) drastično smanjuju pretraživani prostor i sprečavaju zaglavljivanje u lokalnim optimumima na složenim mrežnim strukturama. [3]

2 Opis predloženog rešenja

2.1 Pretraga grubom silom

Pretraga grubom silom (*Brute force*) koristi potpunu pretragu svih mogućih podskupova čvorova. Podskupovi se razmatraju po rastućoj kardinalnosti, počevši od jednog čvora. Za svaki podskup proverava se da li predstavlja dominantni skup, odnosno da li njegova zatvorena okolina pokriva sve čvorove grafa. Pošto se podskupovi proveravaju od najmanjeg ka najvećem, prvi pronađeni dominantni skup je optimalan.

Opis algoritma:

```
Brute_force(G):  
  za r = 1 do |V|:  
    za svaki podskup S veličine r:  
      ako S dominira sve čvorove:  
        vrati S  
  vrati V
```

Algoritam u najgorem slučaju proverava svih $2^{|V|} - 1$ nepraznih podskupova, pa je eksponencijalan. Zato je praktičan samo za relativno male grafove.

2.2 Celobrojno programiranje

Za svaki čvor $v \in V$ uvodi se binarna promenljiva $x_v \in \{0, 1\}$ definisana na sledeći način:

$$x_v = \begin{cases} 1, & \text{ako čvor } v \text{ pripada dominirajućem skupu} \\ 0, & \text{inače} \end{cases}$$

Cilj optimizacije jeste minimizacija ukupnog broja izabranih čvorova, što se matematički zapisuje kao:

$$\min \sum_{v \in V} x_v$$

Da bi skup bio dominirajući, svaki čvor $v \in V$ mora biti pokriven. Pokrivenost podrazumeva da je sam čvor v izabran ili da je izabran bar jedan od njegovih direktnih suseda $u \in N(v)$, gde $N(v)$ označava skup suseda čvora v . Matematička formulacija ograničenja glasi:

$$x_v + \sum_{u \in N(v)} x_u \geq 1, \quad \forall v \in V$$

2.3 Predstavljanje rešenja i funkcije prilagođenosti

Rešenje je predstavljeno pomoću vektora bita dužine $N = |V|$, gde je N ukupan broj čvorova u grafu:

$$\mathbf{x} = [x_0, x_1, \dots, x_{N-1}], \quad x_i \in \{0, 1\}$$

Funkcija prilagođenosti izračunava kvalitet kandidat-rešenja kombinujući broj izabranih čvorova i penal za nepokrivene čvorove:

$$f(\mathbf{x}) = |S| + (N - |C|) \cdot (N + 1)$$

Gde su: $|S|$: Ukupan broj izabranih čvorova, C : Skup izabranih čvorova i svih njihovih direktnih suseda u grafu, $N - |C|$: Broj čvorova u grafu koji nisu dominirani, $N + 1$: Visoka konstanta koja strogo kažnjava nedopustiva rešenja.

2.4 Metoda pretraživanja promenljivih okolina

Za razliku od egzaktnih metoda, metaheuristika Pretraživanje promenljivih okolina (*Variable Neighborhood Search* - *VNS*) koristi se za efikasno pronalaženje zadovoljavajućih rešenja u razumnom vremenskom roku kod NP-teških problema velikih dimenzija.

Metoda promenljivih okolina se sastoji od dva glavna dela: razmrđavanja (*shaking*) (zaduženo za diverzifikaciju) i lokalne pretrage (kao i obično, zadužena za intenzifikaciju). Razmrđavanjem se na nasumičan način dobija novo rešenje koje se nalazi u k -toj okolini trenutnog rešenja. Potom se na novodobijeno rešenje primenjuje lokalna pretraga kako bismo ga unapredili. Ovaj proces se ponavlja dok se ne ispuni kriterijum zaustavljanja.[2]

Kako nasumično potresanje može generisati nedopustivo ili nepovoljno rešenje, primenjuju se specijalizovane operacije:

Popravka (*repair*): Osigurava dopustivost rešenja (ako nakon potresanja neki čvorovi nisu pokriveni, dodaju se novi čvorovi u skup dok se ne zadovolji uslov dominacije).

Orezivanje (*pruning*): Opcionalna faza lokalnog poboljšanja kojom se iz rešenja uklanjaju suvišni čvorovi (čvorovi čijim uklanjanjem skup i dalje ostaje dominirajući), čime se direktno smanjuje vrednost funkcije cilja.

Opis algoritma:

```
VNS(G, numIters, max_k):
    current = best = InicijalnoRešenje(G)
    ponovi numIters puta:
        k = 1
        dok k <= max_k:
            Shake(k) -> Repair -> Prune
            ako fit(cand) < fit(current):
                current = cand
            ako fit(current) < fit(best):
                best = current
                k = 1
            inače:
                k = k + 1
        inače:
            k = k + 1
    vrati best
```

Ukupna vremenska složenost: Za zadati broj iteracija I i maksimalnu veličinu okoline $k_{max} \leq V$:

$$\mathcal{O}(I \cdot k_{max} \cdot V \cdot (V + E))$$

2.5 Genetski algoritam

Genetski algoritam je metaheuristika inspirisana procesom prirodne selekcije i genetikom. Za razliku od VNS-a koji unapređuje jedno rešenje, GA održava i evoluira čitavu populaciju jedinki (kandidat-rešenja) kroz generacije primenom operatora selekcije, ukrštanja i mutacije.

Koristimo turnirsku selekciju gde se iz trenutne populacije nasumično bira podskup od k jedinki:

$$P_{turnir} \subset \mathcal{P}, \quad |P_{turnir}| = k$$

Kao roditelj se bira jedinka sa najboljom (najmanjom) vrednošću funkcije prilagođenosti (*fitness*):

$$p = \min_{x \in P_{turnir}} f(x)$$

Između dva odabrana roditelja p_1 i p_2 bira se nasumična tačka $j \in \{0, 1, \dots, |V| - 1\}$. Generišu se dva potomka c_1 i c_2 kombinovanjem genetskog materijala:

$$c_1 = p_1[0 \dots j - 1] \parallel p_2[j \dots |V| - 1]$$

$$c_2 = p_2[0 \dots j - 1] \parallel p_1[j \dots |V| - 1]$$

Nakon ukrštanja radi se popravka za osiguranje dopustivosti, dok se sa verovatnoćom od 10% vrši orezivanje radi lokalne optimizacije.

Izdvađa se podskup jedinki. Za svaku odabranu jedinku, sa određenom verovatnoćom, nasumično se bira jedan bit i i vrši se njegova inverzija:

$$x[i] = 1 - x[i]$$

3 Rezultati eksperimenta

3.1 Test primeri

Za testiranje je korišćeno više nasumično generisanih grupa neusmerenih bestežinskih grafova:

- Skup 25 malih grafova (5 čvorova) različitih gustina
- Skup 25 srednjih grafova (25 čvorova) različitih gustina
- Skup 25 velikih grafova (100 čvorova) različitih gustina
- Skup grafova od 5 do 210 čvorova prosečne gustine 0.2
- Veliki graf od 1000 čvorova prosečne gustine 200

3.2 Korišćene metrike

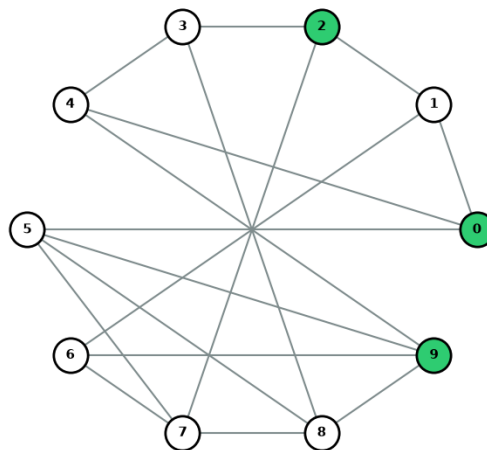
Za testiranje je korišćeno više metrika:

- Prosečna uspešnost (u procentima) - procenat nađenih optimalnih rešenja
- Prosečno odstupanje (u procentima) - procenat udaljenosti od optimalnog rešenja
- Prosečno vreme izvršavanja (u sekundama)

3.3 Ilustrativni primer

Na slici [Slika 1] je prikazan rezultat za Petersenov graf sa 10 čvorova i 15 dobi-
jen primenom celobrojnog programiranja. Dominirajući skup čvorova je obojen zelenom
bojom.

Minimalni dominirajući skup za Petersonov graf



Slika 1: Petersenov graf sa obojenim dominirajućim skupom čvorova

3.4 Eksperimentalno okruženje

Svi testovi i eksperimenti izvršeni su na računarskom sistemu čije su hardverske i softverske specifikacije prikazane u Tabeli 1. Za implementaciju svih predloženih algoritama (Brute Force, ILP, VNS i Genetski algoritam) korišćen je programski jezik Python.

Tabela 1: Specifikacija hardverskog i softverskog okruženja

Komponenta / Parametar	Vrednost / Opis
Procesor (CPU)	Intel(R) Core(TM) i5-11400F 2.60GHz
Radna memorija (RAM)	16.0 GB
Matična ploča	ASRock B560 Steel Legend
Operativni sistem	Microsoft Windows 11 Pro (64-bit)
Programski jezik	Python 3.14.7

3.5 Rezultati na skupovima podataka

Na malom skupu svi algoritmi brzo pronalaze optimalno rešenje u kratkom vremenu. Posebno se po brzini izdvajaju celobrojno programiranje i gruba sila. [Tabela 2]

Tabela 2: Rezultati na malom skupu

Algorithm	Success	Success %	Average Gap %	Average Time (s)
Linear Programming	25	100.0	0.0	3.05×10^{-5}
Brute Force	25	100.0	0.0	0.0074
VNS	25	100.0	0.0	0.0379
VNS (no pruning)	25	100.0	0.0	0.0223
Genetic Algorithm	25	100.0	0.0	0.7077

Na srednjem skupu VNS bez orezivanja već gubi na tačnosti. [Tabela 3]

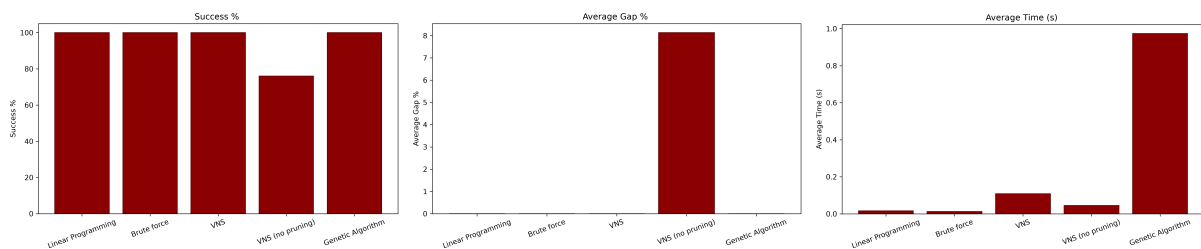
Tabela 3: Rezultati na srednjem skupu

Algorithm	Success	Success %	Average Gap %	Average Time (s)
Linear Programming	25	100.0	0.00	0.0162
Brute Force	25	100.0	0.00	0.0122
VNS	25	100.0	0.00	0.1087
VNS (no pruning)	19	76.0	8.13	0.0448
Genetic Algorithm	25	100.0	0.00	0.9735

Za primere iz velikog skupa algoritam grube sile više nije u mogućnosti da nam da rezultate u razumnom vremenu jer je broj operacija ovde već $2^{100} \approx 1.267 \times 10^{30}$. Ostale metode beleže pad u tačnosti ali su i dalje blizu optimalnog rešenja dok im je vreme izvršavanja značajno poraslo. [Tabela 4]

Tabela 4: Rezultati na velikom skupu

Algorithm	Success	Success %	Average Gap %	Average Time (s)
Linear Programming	25	100.0	0.00	0.1352
Brute Force	/	/	/	~40 milijardi god.
VNS	11	44.0	5.99	12.1996
VNS (no pruning)	0	0.0	27.65	2.3464
Genetic Algorithm	3	12.0	10.39	17.4640



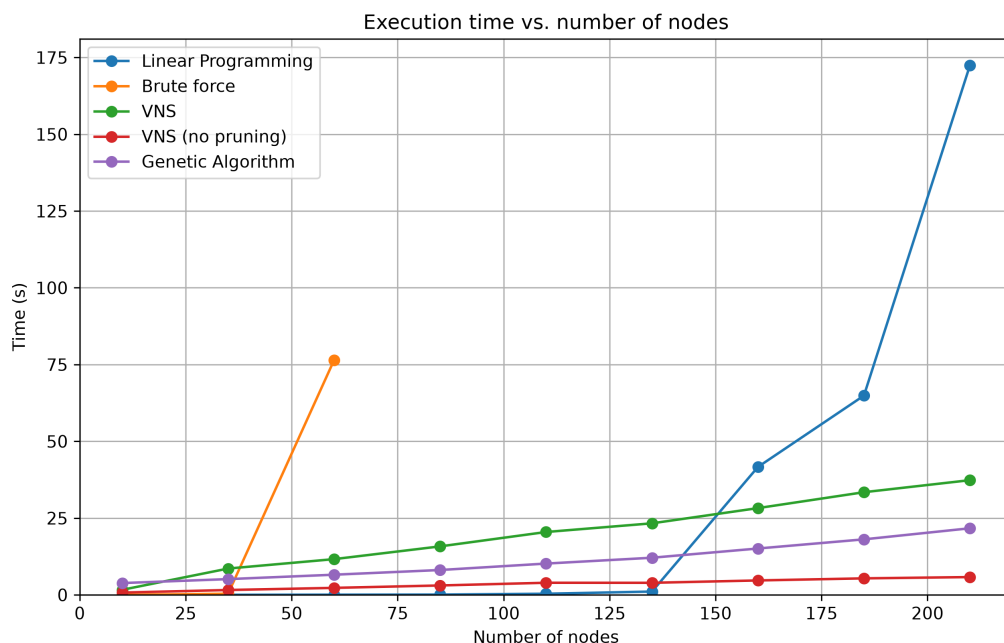
(a) Prosečna uspešnost

(b) Prosešno odstupanje

(c) Prosečno vreme

Slika 2: Vizuelno poređenje rezultata na srednjem skupu

Posmatrajmo vreme izvršavanja u odnosu na broj čvorova u grafu. [Slika 3] Algoritam grube sile brzo postaje neupotrebljiv nakon 60 čvorova, dok celobrojno programiranje doživljava eksploziju oko 210 čvorova. Iako počinju dosta sporije od grube sile i celobrojnog programiranja metaheurističke metode nastavljaju da rastu linearno u odnosu na broj čvorova i ne doživljavaju kombinatornu eksploziju.



Slika 3: Vreme izvršavanja u odnosu na broj čvorova

Linearni rast metaheurističkih metoda nam omogućava da ih primenjujemo na mnogo veće domene. U tabeli je dat primer grafa sa 1000 čvorova prosečne gustine 200

Algorithm	Iterations	Dominating Set Size ($ S $)	Execution Time
VNS	1000	16	608.59 s
VNS	200	16	140.53 s
VNS (no pruning)	1000	19	42.81 s
VNS (no pruning)	10000	18	533.46 s
Genetic Algorithm	1000	16	592.31 s
Genetic Algorithm	200	16	214.26 s

4 Zaključak

4.1 Pregled urađenog

U ovom radu izvršena je analiza i eksperimentalna poređenja različitih pristupa za rešavanje problema minimalnog dominirajućeg skupa. Testirani su egzaktni metodi (pretraga grubom silom i celobrojno linearno programiranje) kao i metaheuristički pristupi (Metoda pretraživanja promenljivih okolina i Genetski algoritam).

Eksperimentalni rezultati jasno pokazuju izražena ograničenja egzaktnih metoda. Pretraga grubom silom pokazuje eksponencijalni rast vremenske složenosti i postaje praktično neprimenjiva porastom broja čvorova. Sa druge strane, metoda celobrojnog programiranja pokazala se izuzetno efikasnom i brzom na malim, srednjim i velikim instancama

do oko 210 čvorova, ali usled kombinatorne eksplozije gubi primenljivost na masivnim grafovima.

4.2 Poređenje metaheuristika

Metaheurističke metode pokazuju svoju pravu snagu na velikim instancama. Posebno se ističe VNS algoritam sa ugrađenim mehanizmom orezivanja (*pruning*). Dok VNS bez orezivanja drastično gubi na tačnosti kako dimenzija problema raste, dodavanje faze orezivanja omogućava efikasno vođenje pretrage i dobijanje rešenja izuzetno bliskih optimalnim. Što je najvažnije, stabilan i skroman rast vremena izvršavanja omogućava primenu VNS algoritma i na grafovima masivnih dimenzija.

Primećujemo da genetski algoritam i VNS sa orezivanjem dosta brzo konvergiraju ka rešenju dok vns bez orezivanja sporo konvergira i daje slabije rezultate. Dok VNS sa orezivanjem i genetski algoritam već pri 200 iteracija brzo konvergiraju ka dominantnom skupu veličine $|S| = 16$ (pri čemu povećanje na 1000 iteracija ne donosi dalja poboljšanja), VNS bez orezivanja ostaje zaglavljen u lošijim lokalnim optimumima

4.3 Mogućnosti za dalji rad

U okviru daljeg istraživanja i unapređenja predloženih rešenja otvara se nekoliko značajnih pravaca:

- **Podešavanje i optimizacija parametara:** Eksperimentalno uporediti uticaj različitih funkcija prilagođenosti i kaznenih faktora za nedopustiva rešenja. Takođe, primena metoda poput mrežne pretrage (*grid search*) za fino podešavanje hiperparametara (veličina populacije, stope ukrštanja i mutacije kod GA, kao i maksimalna veličina okoline kod VNS-a) mogla bi značajno poboljšati brzinu konvergencije.
- **Uvođenje naprednih varijanti grafova:** Proširiti algoritme na složenije strukture grafova, kao što su težinski grafovi (sa težinama na granama), usmereni grafovi (*digraphs*), kao i grafovi sa težinama ili cenama na čvorovima, gde se umesto kardinalnosti minimizuje ukupna cena izabranih čvorova (*Minimum Weight Dominating Set*).
- **Hibridizacija i paralelozacija:** Razmotriti hibridne metaheurističke pristupe (npr. spajanje VNS-a i GA u jedinstven algoritam).

Literatura

- [1] A. Vrdoljak, *Finding a minimal dominating set of a graph combining various heuristic approaches based on variable neighborhood search*, u Proceedings of the Conference on March 14 – International Day of Mathematics, Sarajevo, Bosnia and Herzegovina, Dec. 2024, str. 81–89. Dostupno na: https://www.researchgate.net/publication/387951900_FINDING_A_MINIMAL_DOMINATING_SET_OF_A_GRAPH_COMBINING_VARIOUS_HEURISTIC_APPROACHES_BASED_ON_VARIABLE_NEIGHBORHOOD_SEARCH
- [2] S. Kapunac, *Doktorska disertacija*, Matematički fakultet, Univerzitet u Beogradu. Dostupno na: <http://elibrary.matf.bg.ac.rs/bitstream/handle/123456789/5782/phdStefanKapunac.pdf?sequence=1>
- [3] A. R. Hedar i R. Ismail, *Hybrid Genetic Algorithm for Minimum Dominating Set Problem*, u International Conference on Computational Science and Its Applications (ICCSA), 2010. Dostupno na: https://www.researchgate.net/publication/338340035_Hybrid_Genetic_Alogrithm_for_Minimum_Dominating_Set_Proplem